Constraint-Friendly Map-to-Curve Relations

From the GMMZ Construction to the y-Increment Fix

A Technical Companion

April 21, 2026

Abstract

This document presents a unified technical treatment of two recent papers on map-to-elliptic-curve-group relations for constraint programming. First, we describe the GMMZ construction (Groth, Malvai, Miller, Zhang, Asiacrypt 2025), which replaces the expensive inner cryptographic hash by a nondeterministic map-to-curve relation whose witness is verified by a handful of algebraic constraints. Second, we describe the algebraic attack of El Housni and Bünz, which exploits the order-3 automorphism of $j = 0$ curves (such as BN254) to produce a concrete BLS signature forgery whenever $M \cdot T \gtrsim \sqrt{q}$. Third, we present their countermeasure, the *y-increment* construction, which neutralizes the automorphism by encoding messages into the y-coordinate instead of the x-coordinate, and simultaneously lifts the field restriction $q \equiv 3 \pmod 4$. For each construction we provide the mathematical specification, pseudocode, and a self-contained Go implementation.

Contents

1 Background and Notation **2**
- 1.1 Elliptic curves . . . 2
- 1.2 Constraint programming for zero-knowledge proofs . . . 2

2 The GMMZ x-Increment Construction **2**
- 2.1 Core idea . . . 2
- 2.2 The relation . . . 2
- 2.3 The constructor (prover side) . . . 3
- 2.4 Applications . . . 3
- 2.5 Go implementation . . . 4

3 The Algebraic Attack **6**
- 3.1 The automorphism blind spot . . . 6
- 3.2 Lattice argument . . . 6
- 3.3 The BLS forgery . . . 7
- 3.4 Go implementation of the attack . . . 7

4 The y-Increment Countermeasure **9**
- 4.1 Core idea . . . 9
- 4.2 The relation . . . 10
- 4.3 The constructor . . . 10
- 4.4 Security statement . . . 11
- 4.5 Go implementation . . . 11

5 End-to-End Demo **16**

6 Comparison and Takeaways **19**

1 Background and Notation

1.1 Elliptic curves

Let $q > 3$ be prime and let E be an elliptic curve over $\mathbb{F}_q$ in short Weierstrass form

$$E : y^2 = f(x), \qquad f(x) = x^3 + ax + b.$$

The set $E(\mathbb{F}_q) = \{(x, y) \in \mathbb{F}_q^2 : y^2 = f(x)\} \cup \{\mathcal{O}\}$ forms a finite abelian group, where $\mathcal{O}$ is the point at infinity. The inverse of a point is $-(x, y) = (x, -y)$. A cryptographic subgroup $G \subseteq E(\mathbb{F}_q)$ of prime order p is singled out; the cofactor $h = |E(\mathbb{F}_q)|/p$ is typically small (e.g. $h = 1$ for BN254, secp256k1).

The *j-invariant* classifies a curve up to $\overline{\mathbb{F}_q}$-isomorphism:

$$j(E) = 1728 \cdot \frac{4a^3}{4a^3 + 27b^2}.$$

Curves with $j = 0$ (i.e. $a = 0$, $y^2 = x^3 + b$) admit an extra order-3 automorphism

$$\phi : (x, y) \mapsto (\omega x, y),$$

where $\omega \in \mathbb{F}_q$ is a primitive cube root of unity (existing whenever $q \equiv 1 \pmod 3$). Crucially, $\phi(P) = [\lambda]P$ for a scalar λ with $\lambda^2 + \lambda + 1 \equiv 0 \pmod p$. This structural property is what powers the attack of Section 3.

1.2 Constraint programming for zero-knowledge proofs

In constraint programming one specifies a relation $R \subseteq X \times W$ between public input $x \in X$ and witness $w \in W$ through algebraic equations. A prover demonstrates that it knows w with $(x, w) \in R$ by producing a succinct proof. The cost of proving grows linearly with the number of constraints, so every saved multiplication matters.

Hash-to-curve functions are used for multiset hashing (for zkVM memory consistency) and for BLS signatures (for proof-of-stake aggregation). The classical construction uses an inner cryptographic hash (SHA-256, Poseidon, MiMC) followed by a deterministic map-to-curve. The inner hash dominates the constraint cost: SHA-256 alone costs ~ 7000 constraints per invocation in R1CS, whereas a complete map-to-curve relation can be verified in only a few dozen constraints.

2 The GMMZ x-Increment Construction

2.1 Core idea

GMMZ observes that in the Generic Group Model (GGM), the encoding of a group element already randomizes discrete logarithms. There is therefore no need for an inner cryptographic hash, provided the message space M is small enough that the collision probability $q_{\text{oracle}} \cdot |M|/p$ remains negligible. The map-to-group step alone, expressed as a nondeterministic relation, suffices.

The construction interprets a message $m \in M \subseteq \mathbb{F}_q$ directly as (the base of) an x-coordinate. Since a random x lies on the curve only with probability $\approx 1/(2h)$, an iterative *increment-and-check* is used: the prover tries $x = m \cdot T + k$ for $k = 0, 1, \ldots, T - 1$ and supplies the successful k as witness.

2.2 The relation

Fix an elliptic curve $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_q$ with $q \equiv 3 \pmod 4$, a subgroup $G \subseteq E(\mathbb{F}_q)$ of prime order p, a message space $M \subseteq \mathbb{F}_q$, and a tweak bound $T \in \mathbb{N}$ with $M \cdot T < q$.

Definition 1 (GMMZ x-increment relation)**.** The relation $R_x \subseteq M \times G \times W$ consists of all triples $(m, (x, y), w)$ with $w = (k, z)$ satisfying

$$m \in M, \quad k \in [0, T), \tag{1}$$
$$x = m \cdot T + k, \tag{2}$$
$$z^2 = y, \tag{3}$$
$$y^2 = x^3 + ax + b, \tag{4}$$
$$(x, y) \in G. \tag{5}$$

Role of each constraint.

- (2) **Injectivity.** The multiplier T forces disjoint x-ranges $[mT, (m+1)T)$ for distinct m; given x one recovers (m, k) uniquely by Euclidean division. The side condition $M \cdot T < q$ prevents modular wrap-around.
- (3) **Inverse exclusion.** On an elliptic curve, (x, y) and $(x, -y)$ are both valid points but inverse to each other. Without a canonical choice, two witnesses would map one message to two points. The constraint $z^2 = y$ forces y to be a quadratic residue. When $q \equiv 3 \pmod 4$, -1 is a quadratic non-residue, so exactly one of $\{y, -y\}$ is a QR—canonicality is obtained for free.
- (4),(5) **Curve and subgroup membership.** These ensure (x, y) is actually an element of G.

2.3 The constructor (prover side)

Algorithm 1 GMMZ x-Increment Constructor

Require: curve $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_q$ with $q \equiv 3 \pmod 4$, subgroup G, message $m \in M$, tweak bound T.

```
k ← 0
while k < T do
    x ← m · T + k
    f ← x^3 + ax + b
    if f^((q−1)/2) = 1 in F_q then                ▷ quadratic residue test
        y ← f^((q+1)/4)
        z ← y^((q+1)/4)                           ▷ z^2 = y since y is a QR
        if (x, y) ∈ G then
            return ((x, y), (k, z))
        end if
    end if
    k ← k + 1
end while
return ⊥                                          ▷ failure probability ≈ 2^−T
```

2.4 Applications

Multiset hashing.

Given an injective and inverse-excluding map-to-group relation, define

$$\mathrm{MSH}(S) = \sum_{m \in S} \mathrm{MapToGroup}(m).$$

Incrementality is immediate: adding m to the multiset adds (x_m, y_m) to the digest. Collision resistance holds in EC-GGM with probability at most $(2\sum_m |S_m^{\pm}| \cdot Q + 2Q^2)/p$ for Q oracle queries.

BLS signatures.

- KeyGen: sample $\mathrm{sk} \leftarrow \mathbb{Z}_p$; set $\mathrm{vk} = [\mathrm{sk}]g_2 \in G_2$.
- $\mathrm{Sign}(m)$: compute $(h_m, w_m) \leftarrow \mathrm{Construct}(m)$; output $\sigma = [\mathrm{sk}]h_m$.
- $\mathrm{Verify}(\mathrm{vk}, m, \sigma, w_m)$: check $(m, h_m, w_m) \in R_{\mathrm{M2G}}$ and $e(\sigma, g_2) = e(h_m, \mathrm{vk})$.

2.5 Go implementation

We implement the constructor for BN254. The curve is $y^2 = x^3 + 3$ over $\mathbb{F}_q$ with $q \equiv 3 \pmod 4$, so the square root and QR test are a single exponentiation each.

```go
package mapcurve

import (
    "errors"
    "math/big"

    bn254 "github.com/consensys/gnark-crypto/ecc/bn254"
    "github.com/consensys/gnark-crypto/ecc/bn254/fp"
)

// TweakBound is the maximum number of tweak attempts.
const TweakBound = 256

// XIncWitness is the witness (k, z) for the x-increment relation.
type XIncWitness struct {
    K uint32   // tweak in [0, T)
    Z fp.Element // square root of y: z^2 = y
}

// XIncConstruct maps a message m (given as a field element) to a BN254 G1
// point by the GMMZ x-increment procedure.
//
//   x = m*T + k,    y = sqrt(f(x)),    z = sqrt(y)
//
// Returns the point (x,y) and the witness (k,z). The failure probability
// after T = 256 trials is about 2^{-256}.
func XIncConstruct(m *fp.Element) (bn254.G1Affine, XIncWitness, error) {
    var (
        tBig = big.NewInt(int64(TweakBound))
        mBig = new(big.Int)
        base fp.Element // base = m * T  (as a field element)
    )
    m.BigInt(mBig)
    base.SetBigInt(new(big.Int).Mul(mBig, tBig))

    var (
        x, y, f  fp.Element
        kElement fp.Element
```

```go
    )

    for k := uint32(0); k < TweakBound; k++ {
        // x = m*T + k
        kElement.SetUint64(uint64(k))
        x.Add(&base, &kElement)

        // f = x^3 + ax + b  (for BN254: a=0, b=3)
        computeRHS(&f, &x)

        // QR test: f is a QR iff f^{(q-1)/2} = 1
        if !isQR(&f) {
            continue
        }

        // y = sqrt(f) = f^{(q+1)/4}  (valid because q = 3 mod 4)
        y = sqrtModP3Mod4(&f)

        // z = sqrt(y) = y^{(q+1)/4}  (valid because y is a QR by choice)
        z := sqrtModP3Mod4(&y)

        // Build the affine point and verify subgroup membership.
        var P bn254.G1Affine
        P.X = x
        P.Y = y
        if !P.IsOnCurve() || !P.IsInSubGroup() {
            continue
        }

        return P, XIncWitness{K: k, Z: z}, nil
    }
    return bn254.G1Affine{}, XIncWitness{},
        errors.New("x-increment: no valid tweak found")
}

// computeRHS sets out = x^3 + 3  for BN254 (a=0, b=3).
func computeRHS(out, x *fp.Element) {
    var x2, x3 fp.Element
    x2.Square(x)
    x3.Mul(&x2, x)
    var three fp.Element
    three.SetUint64(3)
    out.Add(&x3, &three)
}

// isQR returns true iff f is a quadratic residue in F_q.
func isQR(f *fp.Element) bool {
    // Legendre symbol: f^{(q-1)/2} = +-1 for nonzero f.
    return f.Legendre() == 1
}

// sqrtModP3Mod4 returns f^{(q+1)/4} when q = 3 mod 4.
func sqrtModP3Mod4(f *fp.Element) fp.Element {
    var out fp.Element
    // gnark-crypto exposes a direct Sqrt; we use it here for brevity.
    out.Sqrt(f)
    return out
```

```
}
```

Listing 1: GMMZ x-increment constructor (Go, BN254).

```go
// XIncVerify re-checks the five algebraic conditions of Definition 1.
// In a SNARK this is compiled to a handful of R1CS constraints.
func XIncVerify(m *fp.Element, P *bn254.G1Affine, w *XIncWitness) bool {
    if w.K >= TweakBound {
        return false
    }

    // Condition (x1): x = m*T + k
    var (
        tElem, kElem, base, x fp.Element
    )
    tElem.SetUint64(uint64(TweakBound))
    kElem.SetUint64(uint64(w.K))
    base.Mul(m, &tElem)
    x.Add(&base, &kElem)
    if !x.Equal(&P.X) {
        return false
    }

    // Condition (x3): z^2 = y  (canonical QR root of y)
    var z2 fp.Element
    z2.Square(&w.Z)
    if !z2.Equal(&P.Y) {
        return false
    }

    // Conditions (x4)-(x5): (x,y) on curve and in subgroup.
    return P.IsOnCurve() && P.IsInSubGroup()
}
```

Listing 2: Verifier-side algebraic check of the x-increment relation.

3 The Algebraic Attack

3.1 The automorphism blind spot

The EC-GGM of Groth–Shoup models the encoding $\pi : \mathbb{Z}_p \to G$ as a random bijection that is identity-preserving and inverse-preserving, but it does not account for the order-3 automorphism ϕ that exists on every $j = 0$ curve over a field with $q \equiv 1 \pmod 3$. The adversary can compute $\phi(P)$ from coordinates alone (one field multiplication: $x \mapsto \omega x$), without issuing any oracle query—yet the security proof only bounds the advantage in terms of oracle queries. This is the model gap.

3.2 Lattice argument

Given $\phi(P_1) = P_2$, the x-coordinates satisfy

$$x_2 \equiv \omega \cdot x_1 \pmod q.$$

The set of all such integer pairs is the lattice

$$L = \{(a, b) \in \mathbb{Z}^2 \,:\, b \equiv \omega \cdot a \pmod q\}, \qquad \det L = q.$$

Minkowski's theorem guarantees a nonzero $(a, b) \in L$ with

$$\max(|a|, |b|) \leq \sqrt{q}.$$

When $M \cdot T > \sqrt{q}$, both $|a|$ and $|b|$ fit inside the encoding range $[0, MT)$. The short vector is computed in $O(\log^2 q)$ time by the extended Euclidean algorithm on (q, ω), or in $O(1)$ closed form for parameterized families.

Proposition 2 (BN short vector, El Housni–Bünz)**.** *For BN curves parameterized by seed u with $q(u) = 36u^4 + 36u^3 + 24u^2 + 6u + 1$, the pair*

$$a(u) = 6u^2 + 2u + 1, \qquad b(u) = 2u$$

satisfies $\omega(u) \cdot a(u) \equiv b(u) \pmod{q(u)}$ with $\omega(u) = 18u^3 + 18u^2 + 9u + 1$, and $\max(|a|, |b|) \leq \sqrt{q}$.

3.3 The BLS forgery

Algorithm 2 Existential BLS signature forgery on a $j = 0$ curve

Require: public key vk, signing oracle $\mathcal{O}_{\text{sk}}$, BN seed u.

Compute (a, b) via the Cornacchia factorization (or Proposition above for BN curves).
$x_1 \leftarrow |a| \bmod q$; let y_1 be the QR root of $x_1^3 + b$.
Let $m_1 \leftarrow \lfloor x_1/T \rfloor$.
Recompute $P_1 = H(m_1)$ by running the public x-increment constructor (the oracle sees the same point by determinism).
Query $\sigma_1 \leftarrow \mathcal{O}_{\text{sk}}(m_1)$.
▷ Apply ϕ^2 as a pure coordinate map on both points.
Set $P_2 \leftarrow (\omega^2 \cdot x(P_1),\ y(P_1))$.
Set $\sigma_2 \leftarrow (\omega^2 \cdot x(\sigma_1),\ y(\sigma_1))$.
Let $m_2 \leftarrow \lfloor x(P_2)/T \rfloor$.
return (m_2, σ_2, P_2).

The forgery bypasses the scalar λ entirely: since ϕ is a group endomorphism, $\phi^2(\sigma_1) = \phi^2([\text{sk}]P_1) = [\text{sk}]\phi^2(P_1) = [\text{sk}]P_2$ purely by group-action linearity. Verification succeeds because

$$e(\sigma_2, g_2) = e([\text{sk}]\phi^2(P_1), g_2) = e(P_2, [\text{sk}]g_2) = e(P_2, \text{vk}).$$

Importantly, computing ϕ^2 is a single field multiplication ($x \mapsto \omega^2 x$) — the adversary never invokes the group addition oracle, which is why the EC-GGM security argument misses this attack entirely.

3.4 Go implementation of the attack

```go
package attack

import (
    "fmt"
    "math/big"

    bn254 "github.com/consensys/gnark-crypto/ecc/bn254"
    "github.com/consensys/gnark-crypto/ecc/bn254/fp"
)

// BN254Seed is the curve-defining seed u (also called x_0). The BN254
```

```go
// moduli are q(u) = 36u^4 + 36u^3 + 24u^2 + 6u + 1, p(u) = q(u) - 6u^2.
var BN254Seed, _ = new(big.Int).SetString(
    "4965661367192848881", 10)

// ComputeShortVector returns (a(u), b(u), omega(u)) from Proposition 1.
//   a(u) = 6u^2 + 2u + 1
//   b(u) = 2u
//   omega(u) = 18u^3 + 18u^2 + 9u + 1   (a primitive cube root of unity in Fq)
func ComputeShortVector(u *big.Int) (a, b, omega *big.Int) {
    u2 := new(big.Int).Mul(u, u)
    u3 := new(big.Int).Mul(u2, u)

    // a = 6u^2 + 2u + 1
    a = new(big.Int).Mul(big.NewInt(6), u2)
    a.Add(a, new(big.Int).Mul(big.NewInt(2), u))
    a.Add(a, big.NewInt(1))

    // b = 2u
    b = new(big.Int).Mul(big.NewInt(2), u)

    // omega = 18u^3 + 18u^2 + 9u + 1
    omega = new(big.Int).Mul(big.NewInt(18), u3)
    omega.Add(omega, new(big.Int).Mul(big.NewInt(18), u2))
    omega.Add(omega, new(big.Int).Mul(big.NewInt(9), u))
    omega.Add(omega, big.NewInt(1))

    return
}

// ForgeBLS produces a valid BLS signature on a fresh message m2 using a
// single signing-oracle query on a crafted m1. Exploits the order-3
// automorphism of BN254 (a j=0 curve).
//
// The trick: phi(x,y) = (omega*x, y) is an automorphism with phi(P) =
// [lambda]P on G1, so applying phi^2 to sigma1 = [sk]*H(m1) as a pure
// coordinate map yields phi^2(sigma1) = [sk]*phi^2(H(m1)) by group-action
// linearity. We never need to know lambda     phi is a coordinate map.
//
// Returns (m2, sigma2, P2), where P2 = phi^2(H(m1)) is the forged hash
// point the verifier will reconstruct. The caller then runs the standard
// BLS pairing check e(sigma2, g2) =? e(P2, vk).
func ForgeBLS(
    signOracle func(m fp.Element) bn254.G1Affine,
) (m2 fp.Element, sigma2 bn254.G1Affine, P2 bn254.G1Affine, err error) {

    aInt, _, omegaInt := ComputeShortVector(BN254Seed)

    // Take x1 = |a| mod q, split into (m1, k1_unused) by x1 = m1*T + k.
```

```go
    qModulus := fp.Modulus()
    x1Int := new(big.Int).Mod(aInt, qModulus)
    T := big.NewInt(int64(TweakBound))
    m1Int := new(big.Int)
    k1Unused := new(big.Int)
    m1Int.DivMod(x1Int, T, k1Unused)

    var m1 fp.Element
    m1.SetBigInt(m1Int)

    // Recompute H(m1) ourselves (public deterministic function); the oracle
    // produces the same point, so we can read its coordinates.
    H1, _, cerr := XIncConstruct(&m1)
    if cerr != nil {
        return fp.Element{}, bn254.G1Affine{}, bn254.G1Affine{},
            fmt.Errorf("XIncConstruct(m1) failed: %w", cerr)
    }

    // Query the signing oracle.
    sigma1 := signOracle(m1)

    // Apply phi^2 as coordinate map: phi^2(x, y) = (omega^2 * x, y).
    var omegaFp, omega2 fp.Element
    omegaFp.SetBigInt(omegaInt)
    omega2.Square(&omegaFp)

    // Forged hash point: P2 = phi^2(H(m1)).
    P2.X.Mul(&omega2, &H1.X)
    P2.Y = H1.Y
    if !P2.IsOnCurve() {
        return fp.Element{}, bn254.G1Affine{}, bn254.G1Affine{},
            fmt.Errorf("P2 off curve (impossible on j=0)")
    }

    // Forged signature: sigma2 = phi^2(sigma1) = [sk] * phi^2(H(m1)).
    sigma2.X.Mul(&omega2, &sigma1.X)
    sigma2.Y = sigma1.Y

    // Extract m2 from P2.x by Euclidean division.
    var x2Big big.Int
    P2.X.BigInt(&x2Big)
    var m2Big, k2Big big.Int
    m2Big.DivMod(&x2Big, T, &k2Big)
    m2.SetBigInt(&m2Big)

    return m2, sigma2, P2, nil
}
```

Listing 3: BN254 algebraic attack: building the short vector and forging a signature.

4 The y-Increment Countermeasure

4.1 Core idea

Since $\phi(x, y) = (\omega x, y)$ *fixes* the y-coordinate, encoding the message into y rather than x makes the automorphism *same-message*: $\phi(P)$ encodes the same m as P, so no cross-message collision

is possible. Two bonus properties fall out:

- **Free inverse exclusion.** The inverse point $-P = (x, -y)$ has y-coordinate $q-y \notin [0, MT)$ whenever $M \cdot T < q/2$. The QR witness chain $z^2 = y$ is no longer needed.
- **No field restriction.** Because inverse exclusion is geometric, the condition $q \equiv 3 \pmod 4$ disappears entirely.

4.2 The relation

Definition 3 (y-increment relation)**.** Let $E : y^2 = x^3 + ax + b$ be an elliptic curve over $\mathbb{F}_q$ with prime subgroup G, message space $M \subseteq \mathbb{F}_q$, and tweak bound T satisfying $M \cdot T < \sqrt{q}$. The relation $R_y \subseteq M \times G \times W$ consists of all $(m, (x, y), w)$ with $w = k$ satisfying

$$m \in M, \quad k \in [0, T), \tag{6}$$
$$y = m \cdot T + k, \tag{7}$$
$$y^2 = x^3 + ax + b, \tag{8}$$
$$(x, y) \in G. \tag{9}$$

Why $M \cdot T < \sqrt{q}$? The tightened bound reflects the true security threshold of *any* such relation in EC-GGM: the Q^2/p birthday term in GMMZ's own theorem caps security at $\sqrt{p}$, which for a prime-order subgroup of a curve over $\mathbb{F}_q$ is essentially $\sqrt{q}$.

4.3 The constructor

Given y, recovering x requires solving the cubic $x^3 + ax + c = 0$ with $c = b - y^2$. Two cases:

$j = 0$ curves ($a = 0$). The cubic degenerates to $x^3 = y^2 - b$, i.e. a cube root. When $q \equiv 1 \pmod 3$, exactly $1/3$ of field elements are cubes, and a candidate y succeeds with probability $\approx 1/3$. With $T = 256$, the failure probability is $(2/3)^{256} < 2^{-150}$.

General curves ($j \neq 0, 1728$). Solve the depressed cubic $x^3+ax+c = 0$ by Cardano's formula, with a Legendre-symbol branch on the discriminant

$$\Delta = -4a^3 - 27c^2.$$

Algorithm 3 y-Increment Constructor (uniform over $j = 0$ and $j \neq 0$)

Require: curve $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_q$, subgroup G, message $m \in M$, tweak bound T.

```
k ← 0
while k < T do
    y ← m · T + k
    c ← b − y^2
    if a = 0 then                                   ▷ j = 0 case: cube root
        x ← (−c)^{1/3} mod q if a cube root exists
    else                                            ▷ general case: Cardano
        x ← root of X^3 + aX + c via Cardano
    end if
    if x ≠ ⊥ and (x, y) ∈ G then
        return ((x, y), k)
    end if
    k ← k + 1
end while
return ⊥
```

4.4 Security statement

Theorem 4 (Informal, El Housni–Bünz). *Let E be a $j = 0$ curve over $\mathbb{F}_q$ with $M \cdot T < \sqrt{q}$. In the standard Groth–Shoup EC-GGM, any adversary making at most Q oracle queries against the y-increment BLS scheme has forging advantage at most*

$$\mathrm{Adv} \leq \frac{c \cdot MT \cdot Q + Q^2}{p}$$

for an absolute constant c. In particular, security reaches the DLP threshold $\min(p/(MT), \sqrt{p})$ without *any model augmentation.*

Conceptually, the proof proceeds exactly like GMMZ's Theorem 3, but now ϕ is harmless because it acts as the identity on the encoding coordinate y. No extra oracle and no extra lattice argument are needed.

4.5 Go implementation

$j = 0$ constructor (BN254).

```go
package mapcurve

import (
    "errors"
    "math/big"
    "sync"

    bn254 "github.com/consensys/gnark-crypto/ecc/bn254"
    "github.com/consensys/gnark-crypto/ecc/bn254/fp"
)

// YIncWitness is the witness k for the y-increment relation.
type YIncWitness struct {
    K uint32
}

// YIncConstructBN254 maps m to a BN254 G1 point using the y-increment
// procedure. For j=0 curves, x-recovery reduces to a cube root.
func YIncConstructBN254(m *fp.Element) (bn254.G1Affine, YIncWitness,
    error) {
    var (
        tBig = big.NewInt(int64(TweakBound))
        mBig = new(big.Int)
        base fp.Element
    )
    m.BigInt(mBig)
    base.SetBigInt(new(big.Int).Mul(mBig, tBig))

    var (
        y, y2, negC, x, kElem fp.Element
    )
    var bFp fp.Element
    bFp.SetUint64(3) // BN254: b = 3

    for k := uint32(0); k < TweakBound; k++ {
        kElem.SetUint64(uint64(k))
        y.Add(&base, &kElem)

```

```go
        // negC = y^2 - b ; we want x^3 = negC
        y2.Square(&y)
        negC.Sub(&y2, &bFp)

        // Cube root in Fq (BN254 has q = 1 mod 3, so only 1/3 of elements
        // are cubes). If negC is not a cube, skip this tweak.
        if !cubeRootInFq(&x, &negC) {
            continue
        }

        var P bn254.G1Affine
        P.X = x
        P.Y = y
        if !P.IsOnCurve() || !P.IsInSubGroup() {
            continue
        }
        return P, YIncWitness{K: k}, nil
    }
    return bn254.G1Affine{}, YIncWitness{},
        errors.New("y-increment: no valid tweak found")
}

// cubeRootInFq computes out = t^{1/3} in BN254's F_q, or returns false if
// t is not a cube.
//
// BN254's q - 1 = 2 * 3^2 * m with gcd(m, 6) = 1 (i.e. v_3(q-1) = 2).
// Therefore 9 | q - 1, and t^{(q+8)/27} cubed equals t * zeta^k for some k,
// where zeta is a primitive 9th root of unity     NOT a mere cube root of
// unity. So the correction loop iterates up to 9 times, not 3.
//
// All arithmetic stays inside gnark-crypto's Montgomery form via Exp/Mul.
var (
    cubeExpOnce  sync.Once
    cubeExpE     *big.Int    // (q+8)/27
    cubeResidueE *big.Int    // (q-1)/3
    zeta9        fp.Element // primitive 9th root of unity in F_q
)

func cubeExpInit() {
    cubeExpOnce.Do(func() {
        q := fp.Modulus()
        cubeExpE = new(big.Int).Add(q, big.NewInt(8))
        cubeExpE.Div(cubeExpE, big.NewInt(27))
        cubeResidueE = new(big.Int).Sub(q, big.NewInt(1))
        cubeResidueE.Div(cubeResidueE, big.NewInt(3))

        // Build zeta9: pick some small g that is NOT a 9th-power residue
        // and set zeta9 = g^{(q-1)/9}.  Require order exactly 9 (so its
        // cube != 1).
        qm1Over9 := new(big.Int).Sub(q, big.NewInt(1))
        qm1Over9.Div(qm1Over9, big.NewInt(9))
```

```go

        var one, g, pow9, cubed fp.Element
        one.SetOne()
        for i := int64(2); i < 1000; i++ {
            g.SetUint64(uint64(i))
            pow9.Exp(g, qm1Over9)
            if pow9.Equal(&one) {
                continue
            }
            cubed.Square(&pow9)
            cubed.Mul(&cubed, &pow9)
            if !cubed.Equal(&one) {
                zeta9 = pow9
                return
            }
        }
        panic("cubeRoot: failed to find a primitive 9th root of unity")
    })
}

func cubeRootInFq(out, t *fp.Element) bool {
    if t.IsZero() {
        out.SetZero()
        return true
    }
    cubeExpInit()

    // Cubic-residue test: t^{(q-1)/3} == 1.
    var r, one fp.Element
    r.Exp(*t, cubeResidueE)
    one.SetOne()
    if !r.Equal(&one) {
        return false
    }

    // Candidate: cand = t^{(q+8)/27}. Then cand^3 = t * zeta9^k for some k.
    var cand, cube fp.Element
    cand.Exp(*t, cubeExpE)

    for i := 0; i < 9; i++ {
        cube.Square(&cand)
        cube.Mul(&cube, &cand)
        if cube.Equal(t) {
            *out = cand
            return true
        }
        cand.Mul(&cand, &zeta9)
    }
    return false
}
```

Listing 4: y-increment constructor for BN254 ($j = 0$).

Verifier.

```go
// YIncVerify re-checks the four algebraic conditions of Definition 3.
// In an R1CS, this compiles to ~16 constraints.
func YIncVerify(m *fp.Element, P *bn254.G1Affine, w *YIncWitness) bool {
    if w.K >= TweakBound {
        return false
    }

    // y = m*T + k
    var tElem, kElem, base, y fp.Element
    tElem.SetUint64(uint64(TweakBound))
    kElem.SetUint64(uint64(w.K))
    base.Mul(m, &tElem)
    y.Add(&base, &kElem)
    if !y.Equal(&P.Y) {
        return false
    }

    return P.IsOnCurve() && P.IsInSubGroup()
}
```

Listing 5: y-increment verifier: only four simple checks, no residuosity chain.

General case: Cardano solver for $j \neq 0, 1728$.

```go
// SolveDepressedCubic tries to find x in F_q with x^3 + a*x + c = 0.
// Returns (x, true) on success, or (_, false) if no F_q-root exists.
//
// Strategy:
//   D^2 = c^2/4 + a^3/27                    (Cardano discriminant term)
//   If D is a square in F_q:
//       u = (-c/2 + D)^{1/3} in F_q
//       x = u - a/(3u)
//   Else:
//       u = (-c/2 + D)^{1/3} in F_{q^2}, and the real root is its trace.
//
// For the y-increment with j != 0, we typically only need the "D-square"
// branch for simplicity. Full generality requires F_{q^2} arithmetic
// (omitted here).
func SolveDepressedCubic(a, c *fp.Element) (fp.Element, bool) {
    var (
        two, three, four, twentySeven fp.Element
        halfC, a3Over27, Dsq          fp.Element
        half                          fp.Element
    )
    two.SetUint64(2)
    three.SetUint64(3)
    four.SetUint64(4)
    twentySeven.SetUint64(27)

    half.Inverse(&two)

    // halfC = c/2
    halfC.Mul(c, &half)

    // a3Over27 = a^3 / 27
```

```go
    var a2, a3 fp.Element
    a2.Square(a)
    a3.Mul(&a2, a)
    var inv27 fp.Element
    inv27.Inverse(&twentySeven)
    a3Over27.Mul(&a3, &inv27)

    // Dsq = (c/2)^2 + a^3/27
    var halfC2 fp.Element
    halfC2.Square(&halfC)
    Dsq.Add(&halfC2, &a3Over27)

    // Only handle the case Dsq is a square in F_q.
    if Dsq.Legendre() != 1 {
        return fp.Element{}, false
    }

    var D fp.Element
    D.Sqrt(&Dsq)

    // inside = -c/2 + D
    var inside fp.Element
    inside.Neg(&halfC)
    inside.Add(&inside, &D)

    var u fp.Element
    if ok := cubeRootInFq(&u, &inside); !ok {
        return fp.Element{}, false
    }

    if u.IsZero() {
        // Degenerate case; root is x = -a/(3u) undefined; use Cardano's
        // depressed limit x = (3c/a) if a != 0.
        if a.IsZero() {
            return fp.Element{}, false
        }
        var x fp.Element
        x.Mul(&three, c)
        var aInv fp.Element
        aInv.Inverse(a)
        x.Mul(&x, &aInv)
        return x, true
    }

    // x = u - a/(3u)
    var threeU, invThreeU, aOver3u, x fp.Element
    threeU.Mul(&three, &u)
    invThreeU.Inverse(&threeU)
    aOver3u.Mul(a, &invThreeU)
    x.Sub(&u, &aOver3u)
    return x, true
}
```

Listing 6: Cardano's depressed-cubic solver over $\mathbb{F}_q$. This is what the y-increment needs when $a \neq 0$.

5 End-to-End Demo

The Go file bundles a `main` function that exercises all three pieces end-to-end on BN254. It (i) runs the x-increment construct and verify on a random 100-bit message, (ii) does the same for the y-increment, and (iii) spins up a toy BLS signing oracle (the key is sampled fresh with `crypto/rand`), asks it for a single signature on a carefully chosen m_1, derives a forged signature on a fresh m_2, and checks the pairing equation to confirm the forgery is accepted.

```go
// BLSOracle is a toy signing oracle: on input m, it runs the x-increment
// constructor to get H(m), then returns [sk] * H(m).
type BLSOracle struct {
    sk      big.Int // secret scalar in [1, p)
    vk      bn254.G2Affine
    g2      bn254.G2Affine
    queried []fp.Element // log of oracle queries
}

func NewBLSOracle() (*BLSOracle, error) {
    _, _, _, g2 := bn254.Generators()

    skInt, err := rand.Int(rand.Reader, fr.Modulus())
    if err != nil {
        return nil, err
    }
    if skInt.Sign() == 0 {
        skInt.SetUint64(1)
    }
    var vk bn254.G2Affine
    vk.ScalarMultiplication(&g2, skInt)
    return &BLSOracle{sk: *skInt, vk: vk, g2: g2}, nil
}

// Sign implements the signing-oracle interface expected by ForgeBLS.
func (o *BLSOracle) Sign(m fp.Element) bn254.G1Affine {
    Hm, _, err := XIncConstruct(&m)
    if err != nil {
        panic(fmt.Sprintf("signing oracle: %v", err))
    }
    o.queried = append(o.queried, m)
    var sigma bn254.G1Affine
    sigma.ScalarMultiplication(&Hm, &o.sk)
    return sigma
}

// pairingCheck returns true iff e(a, b) == e(c, d).
func pairingCheck(a bn254.G1Affine, b bn254.G2Affine,
    c bn254.G1Affine, d bn254.G2Affine) bool {
    var negC bn254.G1Affine
    negC.Neg(&c)
    ok, err := bn254.PairingCheck(
        []bn254.G1Affine{a, negC},
        []bn254.G2Affine{b, d},
    )
    if err != nil {
        return false
    }
    return ok
```

```
}
```

Listing 7: Signing-oracle harness that wraps the x-increment construction into a BLS signer / verifier.

```go
func demoXIncrement() error {
    banner("1) GMMZ x-increment: construct + verify")
    m, err := sampleMessage(100)
    if err != nil {
        return err
    }
    P, w, err := XIncConstruct(&m)
    if err != nil {
        return fmt.Errorf("construct failed: %w", err)
    }
    if !XIncVerify(&m, &P, &w) {
        return errors.New("verifier rejected a valid witness")
    }
    fmt.Println("  verifier accepts")
    return nil
}

func demoYIncrement() error {
    banner("2) y-increment countermeasure: construct + verify (BN254)")
    m, err := sampleMessage(100)
    if err != nil {
        return err
    }
    P, w, err := YIncConstructBN254(&m)
    if err != nil {
        return fmt.Errorf("construct failed: %w", err)
    }
    if !YIncVerify(&m, &P, &w) {
        return errors.New("verifier rejected a valid witness")
    }
    fmt.Println("  verifier accepts")
    return nil
}

func demoAttack() error {
    banner("3) Algebraic attack: BLS signature forgery on x-increment")
    oracle, err := NewBLSOracle()
    if err != nil {
        return err
    }

    // ForgeBLS now returns the forged hash point P2 alongside (m2, sigma2),
    // so no manual reconstruction is needed.
    m2, sigma2, P2, err := ForgeBLS(oracle.Sign)
    if err != nil {
        return fmt.Errorf("forgery failed: %w", err)
    }
    if oracle.AlreadyQueried(m2) {
        return errors.New(
            "ForgeBLS returned the same message it queried     not a real forgery")
```

```go
    }

    if !oracle.VerifyForgedPoint(P2, sigma2) {
        return errors.New("pairing check on forgery FAILED")
    }
    fmt.Println("  pairing check accepts the forgery     BLS is broken")
    return nil
}

func main() {
    fmt.Println("map-to-curve on BN254: demo of GMMZ x-increment,")
    fmt.Println("the El Housni-Bunz forgery, and the y-increment countermeasure.")

    if err := demoXIncrement(); err != nil {
        fmt.Printf("x-increment demo FAILED: %v\n", err)
    }
    if err := demoYIncrement(); err != nil {
        fmt.Printf("y-increment demo FAILED: %v\n", err)
    }
    if err := demoAttack(); err != nil {
        fmt.Printf("attack demo FAILED: %v\n", err)
    }
    banner("done")
}
```

Listing 8: Three demo routines and the `main` driver. Each sub-demo is self-contained.

Running the demo

Expected output (abbreviated), with the concrete forged points elided:

```
map-to-curve on BN254: demo of GMMZ x-increment,
the El Housni-Bunz forgery, and the y-increment countermeasure.

============================================================
  1) GMMZ x-increment: construct + verify
============================================================
  message m = 836...104
  H(m).x    = 213...672
  witness   = (k=1, z=...)
  verifier accepts

============================================================
  2) y-increment countermeasure: construct + verify (BN254)
============================================================
  message m = 475...290
  H(m).y    = 121...648
  witness   = (k=0)
  verifier accepts

============================================================
  3) Algebraic attack: BLS signature forgery on x-increment
============================================================
  forged m2 = 123...
  m2 is distinct from every queried m1
  pairing check accepts the forgery - BLS is broken
```

```
============================================================
  done
============================================================
```

The key observation in line 3 of the output: the verifier accepts a signature on a message the signing oracle was never queried on. This is an existential forgery, the definitional failure of EUF-CMA security.

6 Comparison and Takeaways

	x-increment (GMMZ)	**y-increment (EH-B)**
Encoding coordinate	x	y
Inverse exclusion	QR witness $z^2 = y$	Free (range argument)
Field restriction	$q \equiv 3 \pmod 4$	None
Vulnerable to $j = 0$ automorphism	**Yes**	No (fixed by y)
Security threshold on $M \cdot T$	Was claimed $\ll q$; true is $\sqrt{q}$	$\sqrt{q}$
R1CS constraints (native, BN254)	18	16
R1CS constraints (emulated, BLS12-377)	7741	1065

Practical conclusion. Hashing into a prime-order subgroup of an elliptic curve can indeed be made dramatically constraint-friendly by dropping the inner cryptographic hash. The GMMZ approach correctly captures the main idea, but its security analysis missed the order-3 automorphism on deployed $j = 0$ curves (BN254, BLS12-381, BLS12-377, Grumpkin), resulting in concrete forgeries whenever $M \cdot T \gtrsim \sqrt{q}$. The y-increment fix is both simpler (fewer constraints, no field restriction) and provably secure in the standard EC-GGM, and should be preferred in any deployment.

Where to go from here. For the curious reader: (i) extend the Go implementations above into a full BLS signing/verification pipeline, (ii) compile the verifier of Definition 3 as a gnark circuit and measure constraint counts, (iii) explore the $j = 1728$ case, where the duality remark at the end of §4 shows that x-increment is safe but y-increment is *not*—an interesting reversal of roles.